

NAME: P. Surendranath Reddy

Reg No: 192472356

SUBJECT CODE: CSA0606

SUBJECT NAME: Design and Analysis of Algorithms

Q23. Application: Weather Forecasting Simulation System

```
#include <stdio.h>
#include <math.h>

void algorithm1(int n) {
    for (int i = 1; i <= n; i *= 2) {
        for (int j = 1; j <= n; j *= 2) {
            // Weather simulation operation
        }
    }
}

void algorithm2(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 1; j <= n; j *= 2) {
            // Weather simulation operation
        }
    }
}

int main() {
    int n = 100;
    algorithm1(n);
    algorithm2(n);
    printf("Weather Forecasting Simulation Completed\n");
    return 0;
}
```

Report

Problem:

A weather forecasting system compares two algorithms for simulating weather data: Algorithm 1: $O(\log^2 n)$, and Algorithm 2: $O(n \log n)$. The performance of both algorithms is compared for large-scale weather data.

Complexity Analysis:

Algorithm 1: $O(\log^2 n)$

The outer loop runs approximately $\log n$ times. The inner loop also runs approximately $\log n$ times. Therefore: $\log n \times \log n = \log^2 n$.

Time Complexity: $O(\log^2 n)$

Algorithm 2: $O(n \log n)$

The outer loop runs n times. The inner loop runs $\log n$ times. Therefore: $n \times \log n = n \log n$.

Time Complexity: $O(n \log n)$

Comparison:

$O(\log^2 n) < O(n \log n)$

For large values of n , $O(\log^2 n)$ grows much more slowly than $O(n \log n)$.

Conclusion:

The $O(\log^2 n)$ algorithm is more efficient for large-scale weather data because its growth rate is much smaller. The $O(n \log n)$ algorithm takes more time as the amount of weather data increases. Therefore, Algorithm 1 is the better choice for scalability and large datasets.

Output:

```
$ ./weather_forecasting  
Weather Forecasting Simulation Completed
```

Q63. Application: Digital File Search System

```
#include <stdio.h>

int binarySearch(int arr[], int n, int key) {
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main() {
    int files[] = {10, 20, 30, 40, 50, 60, 70, 80};
    int n = 8;
    int key = 50;

    int result = binarySearch(files, n, key);

    if (result != -1)
        printf("File Found at position %d", result);
    else
        printf("File Not Found");

    return 0;
}
```

Report

Problem:

A digital file search system repeatedly reduces the search space by half while locating a file in a sorted directory. This process is called Binary Search.

Algorithm:

- Set low = 0.
- Set high = n - 1.
- Find the middle element.
- Compare the middle element with the required file.
- If the file is found, return its position.
- If the key is greater, search the right half.
- If the key is smaller, search the left half.
- Repeat until the file is found or the search space becomes empty.

Number of Steps:

At every step, the search space becomes half: $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

After k steps: $n / 2^k = 1$

Therefore: $k = \log_2 n$

Time Complexity:

Best Case: $O(1)$

Average Case: $O(\log n)$

Worst Case: $O(\log n)$

Space Complexity:

The iterative binary search uses only a few variables: low, high, and mid. Therefore, the space complexity is $O(1)$.

Final Complexity:

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Explanation:

- The search space is reduced by half after every comparison.
- Therefore, the number of operations increases logarithmically.
- Even when the number of files becomes very large, the search remains efficient.

Conclusion:

Binary search is highly efficient for searching files in a sorted directory. Its logarithmic time complexity makes it suitable for large digital file systems.

Output:

```
$ ./digital_file_search  
File Found at position 4
```

Time Complexity = $O(\log n)$

Space Complexity = $O(1)$